

HEALTHCARE MANAGEMENT PLATFORM

MILESTONE DOCUMENTATION

Clinical Monitoring, Alert Management & Anomaly Detection Components

Component Scope

1. Vitals Range Validation
2. Alert Fatigue Prevention
3. Anomaly Detection Precision

Prepared for internal engineering review

Table of Contents

Table of Contents	2
1. Vitals Range Validation	4
1.1 Overview of Vitals Range Validation	4
1.2 Role in the Platform	4
1.3 REST API Design	4
1.4 Input Parameters.....	4
1.5 Two Levels of Validation.....	5
1.5.1 Input Validation.....	5
1.5.2 Vital Status Classification.....	5
1.6 Configured Clinical Ranges	5
1.7 Vitals Data Model.....	5
1.7.1 Why Status Is Stored, Not Just Raw Values.....	6
1.8 Complete Vitals Data Flow	6
1.8.1 Example Transaction — Patient P101	6
1.8.2 Responsibility Boundaries	6
1.9 Patient History Sub-Component.....	6
1.10 Technical Significance	7
2. Alert Fatigue Prevention	8
2.1 Overview of Alert Fatigue Prevention	8
2.2 The Core Problem.....	8
2.3 Alert Categories.....	8
2.4 Suppression Rate Calculation	8
2.5 Component Flow	9
2.6 Dashboard Integration	9
2.7 Technical Significance.....	9
3. Anomaly Detection Precision	10
3.1 Overview of Anomaly Detection	10
3.2 Rule-Based Validation vs. ML-Based Detection	10
3.3 Isolation Forest Algorithm.....	10
3.4 Detection Flow	10
3.5 API Integration Output.....	10
3.6 Precision Calculation and Threshold.....	11
3.7 Integrated Dashboard	11
3.8 Technical Significance.....	11
4. Cross-Component Integration Summary	12

4.1 How the Three Components Relate..... 12

4.2 API Endpoints Summary 12

4.3 Dashboard Indicators 12

4.4 Technical Significance of This Milestone 12

1. Vitals Range Validation

1.1 Overview of Vitals Range Validation

Vitals Range Validation is the entry-point component of the Healthcare Management Platform's clinical monitoring pillar. Its responsibility is to accept patient physiological measurements, confirm that each submitted value is structurally acceptable, and classify the accepted value against a configured clinical operating range. This component ensures that every vital sign entering the system is both well-formed and clinically interpretable before it is stored or surfaced to a clinician.

Unlike a generic form-submission endpoint, this component treats each vital sign as a distinct clinical parameter with its own expected physiological range. A single out-of-range reading can be clinically significant, so the platform does not merely store raw numbers — it computes and persists a status alongside every value.

1.2 Role in the Platform

Vitals Range Validation sits directly beneath the dashboard's vital-entry form and above the persistence layer. It is the boundary through which all physiological data must pass before becoming part of a patient's clinical record. Because the frontend never writes to the database directly, this component is the sole authority for deciding whether a reading is LOW, NORMAL, or HIGH.

This positioning is deliberate. It guarantees that classification logic exists in exactly one place, so the same heart rate value is always classified the same way regardless of which page or client requested it.

1.3 REST API Design

Method	Endpoint	Purpose
POST	/api/vitals	Submit a new vital reading for a patient
GET	/api/getvitals	Retrieve stored vital records, including full history

The POST endpoint accepts a structured payload representing one observation for one patient. The GET endpoint returns previously persisted records, enabling both the current-status view and the historical trend view described in Section 1.9.

1.4 Input Parameters

Each submission carries a Patient ID together with six physiological parameters:

- Heart Rate
- SpO₂ (blood oxygen saturation)
- Temperature
- Respiratory Rate
- Systolic Blood Pressure
- Diastolic Blood Pressure

1.5 Two Levels of Validation

1.5.1 Input Validation

Input validation is implemented using Jakarta Bean Validation annotations (`@NotBlank`, `@NotNull`, `@Min`, `@Max`) at the request boundary. This layer only determines whether a request is technically well-formed — for example, an SpO₂ value of 150 is rejected outright because it exceeds the valid input constraint, independent of any clinical interpretation.

1.5.2 Vital Status Classification

Once a request passes input validation, the service layer compares each accepted value against a configured clinical operating range and assigns a status. For example, Heart Rate below 60 is classified LOW, 60–100 is NORMAL, and above 100 is HIGH.

An important technical distinction is that input validation and clinical classification are separate concerns. Input validation checks whether the request itself is acceptable; the service layer separately determines whether an accepted physiological value is LOW, NORMAL, or HIGH according to the configured ranges.

1.6 Configured Clinical Ranges

Vital Sign	Normal Range	Unit
Heart Rate	60 – 100	bpm
SpO ₂	95 – 100	%
Temperature	36.1 – 37.2	°C
Respiratory Rate	12 – 20	/min
Systolic BP	90 – 120	mmHg
Diastolic BP	60 – 80	mmHg

1.7 Vitals Data Model

The persisted entity mirrors the classification design: every raw value is stored alongside its computed status field, and a timestamp is assigned automatically.

Field	Type
id	Primary Key
patientId	String
heartRate / heartRateStatus	Numeric / Enum (LOW, NORMAL, HIGH)
spo2 / spo2Status	Numeric / Enum
temperature / temperatureStatus	Numeric / Enum
respiratoryRate / respiratoryRateStatus	Numeric / Enum

Field	Type
systolicBp, diastolicBp / bloodPressureStatus	Numeric / Enum
recordedAt	Timestamp (@PrePersist)

1.7.1 Why Status Is Stored, Not Just Raw Values

Storing only the raw measurement (for example, Heart Rate = 115) would force every downstream consumer to independently re-implement the classification logic on every retrieval, risking inconsistent interpretation across the dashboard, history view, and any future client. By persisting Heart Rate = 115, Status = HIGH as part of the record, classification becomes part of the clinical record itself and is retrieved consistently every time.

1.8 Complete Vitals Data Flow

Submit Form
POST /api/vitals
Input Validation
Process & Classify
Persist & Respond

1.8.1 Example Transaction — Patient P101

Vital	Value	Classification
Heart Rate	115 bpm	HIGH
SpO ₂	92%	LOW
Temperature	38.2 °C	HIGH
Respiratory Rate	24 /min	HIGH
Blood Pressure	145/95 mmHg	HIGH

1.8.2 Responsibility Boundaries

- Frontend — initiates the transaction only; captures form data and sends the HTTP request.
- Backend — owns validation, classification, and persistence; all business logic lives here.
- Database — stores the complete record, including both raw values and computed status fields.
- Response — JSON returned to the frontend, displayed as the current reading and appended to history.

1.9 Patient History Sub-Component

A single measurement provides only a snapshot; multiple measurements form a time series that gives far more clinical context than the latest value alone. The history sub-component retrieves stored records via GET `/api/getvitals`, filters by Patient ID, sorts chronologically, and renders the sequence as a table and trend chart.

Time	Heart Rate
08:00	72 bpm
10:00	78 bpm
12:00	91 bpm
14:00	105 bpm
16:00	115 bpm (rising)

History information displayed includes Patient ID, all six vital values, each vital's status, the recorded timestamp, and the full historical measurement sequence.

1.10 Technical Significance

Vitals Range Validation establishes the data-quality boundary of the platform. It ensures that a physiological measurement is never treated as clinically meaningful merely because it was numerically parseable. By separating structural validation from clinical classification and persisting both the raw value and its status, the component gives every downstream consumer — the dashboard, the alert engine, and the anomaly-detection service — a consistent, pre-classified view of patient vitals.

2. Alert Fatigue Prevention

2.1 Overview of Alert Fatigue Prevention

Alert Fatigue Prevention addresses a well-documented risk in clinical monitoring software: when the same underlying condition repeatedly triggers notifications, clinicians can become desensitized and begin ignoring alerts altogether, including clinically important ones. This component does not decide whether a vital reading is abnormal — that determination is made upstream by Vitals Range Validation — it instead governs how repeated abnormal events are communicated so that clinician attention is not overwhelmed.

2.2 The Core Problem

Same Condition
Multiple Events
Multiple Notifications
Alert Overload

If every repeated out-of-range reading for the same underlying condition were surfaced as an independent notification, the volume of alerts would rapidly exceed what a clinician can meaningfully act on. The platform therefore introduces a suppression layer between raw alert generation and what is actually displayed to the clinician.

2.3 Alert Categories

Category	Definition
Total Alerts	All alerts generated within the monitored period
Suppressed Alerts	Alerts filtered or grouped by the alert-processing logic
Active Alerts	Alerts that remain visible for clinician attention

2.4 Suppression Rate Calculation

Suppression Rate = (Suppressed Alerts / Total Alerts) × 100

Total Alerts	Suppressed	Active	Suppression Rate
100	60	40	60%

In this example, out of 100 alerts generated, 60 are suppressed as repetitive and 40 remain active for clinician review, giving a suppression rate of 60%. This metric itself is displayed on the dashboard so that clinicians and administrators can gauge how much alert volume is being filtered at any given time.

2.5 Component Flow

Alert Data
Alert API
Alert Processing
Suppression Decision → Suppressed / Active
Dashboard

2.6 Dashboard Integration

The Alert Fatigue Prevention page surfaces alert statistics, the current suppression rate, the count of active alerts, the count of suppressed alerts, and recent alert information. These figures also feed into the platform's integrated dashboard indicator for “Alerts Suppressed,” giving a single at-a-glance view of alert-management effectiveness alongside vitals and anomaly indicators.

2.7 Technical Significance

This component focuses on the usability of clinical monitoring rather than on detection accuracy itself. If every repeated event became a separate notification, clinicians could be overwhelmed to the point of missing genuinely important alerts. By making suppression visible and measurable — rather than silent — the component gives the platform accountability for how much alert volume is being filtered, and preserves clinician trust in the alerts that do reach them.

3. Anomaly Detection Precision

3.1 Overview of Anomaly Detection

Anomaly Detection Precision is the machine-learning component of the platform, responsible for identifying patient observations that appear statistically unusual even when no single vital sign has crossed a configured threshold. It complements, rather than replaces, Vitals Range Validation: a patient's overall pattern of readings can be unusual in combination even if each individual value is technically within range.

3.2 Rule-Based Validation vs. ML-Based Detection

Dimension	Vitals Validation	Anomaly Detection
Approach	Predefined rules	ML model
Question asked	Does HR > 100?	Is this pattern unusual?
Thresholds	Known, configured	Learned from data
Output	LOW / NORMAL / HIGH	Anomaly score + prediction

The anomaly component should not be confused with the vitals validation component. The vitals module answers whether a measurement crosses a configured threshold. The anomaly model answers whether the overall observation appears unusual compared with the patterns learned by the model.

3.3 Isolation Forest Algorithm

The integrated anomaly service uses an Isolation Forest-based approach, an unsupervised anomaly detection technique that does not require labeled examples of abnormal patients to function.

Normal Data Behavior	Anomalous Data Behavior
Normal observations cluster together and require many splits to become isolated from the rest of the dataset.	Unusual observations are already isolated from the cluster and require fewer splits to become isolated — this is the detection signal.

3.4 Detection Flow

Patient Data
Anomaly API (configured endpoint)
Isolation Forest Model
Prediction + Anomaly Score
Frontend Display

3.5 API Integration Output

Each call to the anomaly service returns:

- Prediction (anomalous / not anomalous)
- Anomaly status
- Anomaly score
- Request status
- Precision metrics

3.6 Precision Calculation and Threshold

Precision = $TP / (TP + FP)$, where TP = True Positives and FP = False Positives.

True Positives	False Positives	Precision
90	10	90%

The dashboard applies an 85% precision floor to its monitoring view: a precision of 85% or above is treated as Meets, while precision below 85% is flagged for Review. This threshold gives the platform an explicit, auditable standard for when the anomaly model's output should be trusted at face value versus re-examined.

3.7 Integrated Dashboard

The anomaly-detection-precision page displays model status, the current prediction, the anomaly score, true positive and false positive counts, precision, and a precision trend chart. This feeds the platform-wide dashboard indicator “Anomaly Precision” alongside Vitals Out of Range and Alerts Suppressed, giving a single consolidated view of all three components covered in this milestone.

FRONTEND
VITALS ALERTS ANOMALY
Spring Boot Alert API Isolation Forest
PostgreSQL / Prediction & Precision
DASHBOARD → CLINICIAN

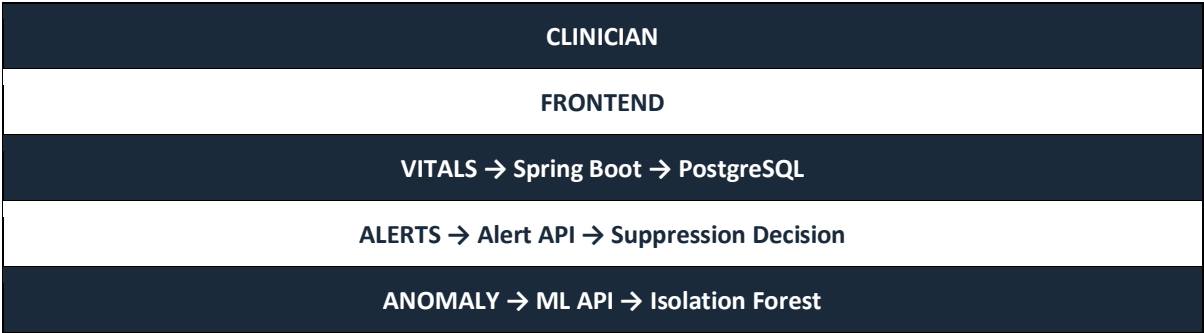
3.8 Technical Significance

Anomaly Detection Precision extends the platform's monitoring capability beyond fixed clinical thresholds into pattern-based detection, while the precision floor and true/false-positive reporting keep the model's output accountable rather than treated as an unquestioned black box. Together with rule-based vitals validation, it gives the platform two complementary lenses on the same patient data: one grounded in known clinical ranges, and one grounded in learned statistical patterns.

4. Cross-Component Integration Summary

4.1 How the Three Components Relate

Although Vitals Range Validation, Alert Fatigue Prevention, and Anomaly Detection Precision are documented individually, they form a single monitoring pipeline rather than three independent features. A vital reading is first validated and classified; if it is out of range, it can generate an alert, which the Alert Fatigue Prevention component then decides to surface or suppress; independently, the same underlying patient data is evaluated by the Isolation Forest model for pattern-level anomalies. All three outcomes converge on the same clinician-facing dashboard.



4.2 API Endpoints Summary

Component	Endpoint(s)	Method
Vitals Range Validation	/api/vitals, /api/getvitals	POST, GET
Alert Fatigue Prevention	Alert API (suppression statistics)	GET
Anomaly Detection Precision	Anomaly API (configured endpoint)	POST/GET

4.3 Dashboard Indicators

- Vitals Out of Range — sourced from Vitals Range Validation
- Alerts Suppressed — sourced from Alert Fatigue Prevention
- Anomaly Precision — sourced from Anomaly Detection Precision
- Module Status — aggregate health of all three components

4.4 Technical Significance of This Milestone

Together, these three components give the Healthcare Management Platform a layered approach to patient safety: deterministic, clinically-defined thresholds catch known danger zones; suppression logic keeps that information usable rather than overwhelming; and unsupervised anomaly detection catches patterns that fixed thresholds alone would miss. This separation of concerns — validation, alerting, and detection — mirrors the platform's broader service-oriented design and allows each concern to be refined independently as clinical requirements evolve.